



University of
Applied Sciences

TECHNIKUM

WIEN

Process management

Process

- **A process is a running instance of a program.**
- **Processes are a fundamental abstraction of the OS (similar to files).**
- **Each instance has its own address space for its execution state.**
- **Each process has a unique PID (process id).**
- **The OS uses this information to allocate and manage resources.**

- **States:**
 - **Running: currently executed (or scheduled)**
 - **Sleeping: ready but inactive**
 - **Interruptable, Uninterruptable, Killable: wait for event (e.g.: I/O-Operation)**
 - **Stopped: not ready; requires SIGCONT retrieval for further execution**
 - **Zombie: terminated, but resources not released**

Process management

- **The OS manages the „process context“ in the „process table“**
 - **PC and SP (Program counter, stack Pointer)**
 - **State of his data/stack segment**
 - **State of hardware register**
 - **State of the data structures of the process**
- **Linux stores entries for:**
 - **Process management**
 - **Memory management**
 - **File management**

Process attributes

Process	Memory	File
• Register content • PC • PSW (program status word) • SP (stack pointer) • Process state • Creation timestamp • Used CPU-time • Used CPU-time of childprocesses • Pointer on messages • PID (Process ID) • Priority	• Pointer to CodeSegment • Pointer to DataSegment • Real UID (user identification) • Effective UID • Real GID (group identification) • Effective GID	• Working directory • Root directory • umask bits • File Pointer

Related C-Functions

- **PID – getpid()**
- **PPID (Parent PID) – getppid()**

```
1 #include <stdio.h>
2 #include <sys/types.h>
3 #include <unistd.h>
4
5 int main () {
6     pid_t pid,ppid;
7     pid=getpid(); /* own process id */
8     ppid=getppid(); /* parent process id */
9     printf ("The process ID is %d\n",(int)pid);
10    printf ("The parent process ID is %d\n",(int)ppid);
11    return 0;
12 }
```



University of
Applied Sciences

TECHNIKUM

WIEN

Child Process

fork()

- A new related (child) process is created with fork()
- It is an exact copy except the PID (PPID)
 - Including descriptors
- Both processes continue the execution flow
- own data- and stack segment
- tree structure (see: [pstree](#))

```
1 #include <sys/types.h>
2 #include <unistd.h>
3 #include <stdlib.h>
4
5 int main()
6 {
7     pid_t pid;
8     pid = fork();
9     switch (pid)
10    {
11        case -1: /* error */
12            return EXIT_FAILURE;
13        case 0: /* child process */
14            break;
15        default: /* parent */
16            break;
17    }
18    return EXIT_SUCCESS;
19 }
```

(simple) example

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/types.h>
4
5 int main(void) {
6     pid_t childpid;
7     childpid = fork();
8     if (childpid == -1) {
9         perror("Failed to fork");
10        return 1;
11    }
12    if (childpid == 0) /* child code */
13        printf("I am child %ld\n", (long)getpid());
14    else /* parent code */
15        printf("I am parent %ld\n", (long)getpid());
16
17    return 0;
18 }
```

(complex) example

```
1 #include <unistd.h>
2 #include <sys/types.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 int global_var = 1;
7
8 int main (void) {
9     pid_t pid;
10    int local_var = 1;
11
12    switch (pid = fork ()) {
13        case -1:
14            fprintf (stderr, "error in fork()\n");
15            return EXIT_FAILURE;
16
17        case 0:
18            sleep (1);
19            printf ("--- child process ---\n");
20            printf ("global_var = %d memory address : %p\n", global_var, &global_var);
21            printf ("local_var = %d memory address : %p\n", local_var, &local_var);
22            fflush(stdout);
23
24            ++global_var; ++local_var;
25            printf ("--- child process ---\n");
26            printf ("global_var = %d memory address : %p\n", global_var, &global_var);
27            printf ("local_var = %d memory address : %p\n", local_var, &local_var);
28            fflush(stdout);
29
30            break;
31
32        default:
33            printf ("--- parent process ---\n");
34            printf ("global_var = %d memory address : %p\n", global_var, &global_var);
35            printf ("local_var = %d memory address : %p\n", local_var, &local_var);
36            fflush(stdout);
37
38            sleep (2);
39            printf ("--- parent process ---\n");
40            printf ("global_var = %d memory address : %p\n", global_var, &global_var);
41            printf ("local_var = %d memory address : %p\n", local_var, &local_var);
42            fflush(stdout);
43
44            break;
45    }
46
47    return EXIT_SUCCESS;
48 }
```

Wait

- **pid_t wait(int *stat_loc);**
 - When child-process terminates, its pid will be returned
 - Else -1 will be returned and errno is set

waitpid

- **pid_t waitpid(pid_t pid, int *stat_loc, int options);**
 - **Enables to wait for a specific pid (or all if pid == -1)**
 - **use NoHang - Option**
 - returns 0 in case a pid is set, but the corresponding process is not yet terminated
 - **Errors:**
 - ECHILD: The process specified by *pid* does not exist or is not a child of the calling process.
 - EINTR: WNOHANG was not set and an unblocked signal or a SIGCHLD was caught

Best Practice to prevent zombie-processes.

- The following code snippets waits for all child-processes that are already terminated, but do not block in case of other child-processes.

```
1  pid_t childpid;
2  while(childpid = waitpid(-1, NULL, WNOHANG)) {
3      if ((childpid == -1) && (errno != EINTR)) {
4          break;
5      }
6  }
```

Macros for return-status checking of child-processes

- **#include <sys/wait.h>**
- **WIFEXITED(int stat_val)**
- **WEXITSTATUS(int stat_val)**
- **WIFSIGNALED(int stat_val)**
- **WTERMSIG(int stat_val)**
- **WIFSTOPPED(int stat_val)**
- **WSTOPSIG(int stat_val)**

Macro-usage

- **Used in pairs: if WIFEXITED returns true and the child process executed its tasks normally (no error, no signal) then the result-state (max 8 bit) can be queried using WEXITSTATUS**
- **The other macros are used when the process was stopped or caused by a signal.**

(complex) example

```
1 #include <errno.h>
2 #include <stdio.h>
3 #include <sys/types.h>
4 #include <sys/wait.h>
5
6 void show_return_status(void) {
7     pid_t childpid;
8     int status;
9
10    childpid = wait(&status);
11    if (childpid == -1)
12        perror("Failed to wait for child");
13    else if (WIFEXITED(status) && !WEXITSTATUS(status))
14        printf("Child %ld terminated normally\n", (long)childpid);
15    else if (WIFEXITED(status))
16        printf("Child %ld terminated with return status %d\n", (long)childpid, WEXITSTATUS(status));
17    else if (WIFSIGNALED(status))
18        printf("Child %ld terminated due to uncaught signal %d\n", (long)childpid, WTERMSIG(status));
19    else if (WIFSTOPPED(status))
20        printf("Child %ld stopped due to signal %d\n", (long)childpid, WSTOPSIG(status));
21 }
```

exec()

- **Many applications require that child-processes execute different code than the parent-process (in contrary to fork())**
- **The exec-functions allows to use a different process image than the calling one right from the beginning.**
 - **There shall be no return from a successful exec, because the calling process image is overlaid by the new process image.**

exec

- The l-named functions use lists of arguments
- The v-named functions use argv (using makeargv)
- The p-named functions use the path variable
- The e-named functions allow to define an „own“ environment

```
#include <unistd.h>

extern char **environ;

int execl(const char *pathname, const char *arg, ...
          /*, (char *) NULL */);
int execlp(const char *file, const char *arg, ...
          /*, (char *) NULL */);
int execlx(const char *pathname, const char *arg, ...
          /*, (char *) NULL, char *const envp[] */);
int execv(const char *pathname, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execvpe(const char *file, char *const argv[], char *const envp[]);
```

<https://man7.org/linux/man-pages/man3/exec.3.html>

execv

- **The execv function executes**
 - **a program (pathname)**
 - absolute path
 - **with arguments (argv)**
 - argv[0] is the program name
 - last argument needs to be NULL ((const char *)0)
- **The execl function is equal to execv but uses a list of arguments.**
- **The execvp/execlp uses additionally the \$PATH to find the program.**

execl()

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main(void) {
7     pid_t childpid;
8
9     childpid = fork();
10    if (childpid == -1) {
11        perror("Failed to fork");
12        return 1;
13    }
14
15    if (childpid == 0) { /* child code */
16        execl("/bin/ls", "ls", "-l", NULL);
17        perror("Child failed to exec ls");
18        return 1;
19    }
20
21    if (childpid != wait(NULL)) { /* parent code */
22        perror("Parent failed to wait due to signal or error");
23        return 1;
24    }
25
26    return 0;
27 }
```

Program attributes in exec-functions

- **Many attributes will be inherited**
 - **PID**
 - **UID**
 - **working directory**
 - **open file descriptors**